总体结论

系统	makespan	相对 baseline	accuracy
H100 baseline（HF transformers，逐题串行）	1470.6s	1×	0.937
H100 Phase 1（单请求引擎优化）	457.4s	3.2×	0.945
H100 Phase 2（多 agent 并发）	18.19s	81×	0.945
H100 Phase 3（自写引擎 + kernel）	8.36s	176×	0.945
Ascend baseline（HF/torch-npu eager serial）	3466.21s	1×	0.9449
Ascend Phase 1（StaticCache + NPU eager serial）	3552.29s	0.98×	0.9449
Ascend Phase 2（16-slot eager batching）	656.37s	5.3×	0.9449
Ascend Phase 3 system（prefix/state reuse + forced suffix）	556.76s	6.2×	0.9449

> H100 Phase 1 的 457.4s 与早期报告中的 474.8s 差异来自 workload 口径不同：前者排除了 task 423，后者包含全部 128 个任务。后续平台对比统一采用排除 task 423 的 127-task 结果。

本周分别在 H100 和 Ascend 910B 上完成了三阶段端到端优化实验。

H100 通过 CUDA 生态和自研推理引擎，将 makespan 从 1673.6s 降至 8.36s。Ascend 910B 则围绕 NPU/CANN 特性进行了独立优化，包括 StaticCache、slot batching、prefix reuse、forced token、NPU layout、Ascend C 和 GDN/conv 路径探索。

Ascend 当前最佳完整结果约为 656.4s。虽然部分局部路径获得收益，但尚未形成稳定的系统级加速。

本周的核心结论是：

> Ascend 当前的问题不是简单照搬 H100，也不是缺少单个优化开关，而是已经测试的 NPU 侧优化主要集中在局部 cache、batching、layout 和算子路径，尚未同时解决 eager launch、prefill、GDN 状态更新、layout transform 和 GEMM 等主要端到端瓶颈。

H100 的结果作为对照，用于帮助识别性能差距和瓶颈来源，并不代表 Ascend 方案直接复制了 H100 的实现。

———

Phase 1 结论

Ascend Phase 1 已经测试了：

- StaticCache；
- NPU internal format；

- MatMul ND/NZ layout；
- async/chunked readback；
- decode input 复用；
- compile/backend 可行性；
- NPU profile。

127-task 严格结果：

DynamicCache：3466.2s StaticCache：3552.3s accuracy：0.9449，两者一致 turns/tool calls：一致

StaticCache 慢 2.48%，因此没有形成有效端到端收益。

原因不是 StaticCache 方向错误，而是当前 Ascend 上 StaticCache 没有与有效 graph replay、稳定 shape 和低成本 mask 路径结合。单独预分配 cache 反而增加了部分执行成本。

Phase 1 结论：完成了候选筛选，但当前软件栈没有满足稳定 5% 加速门槛的方案。

———

Phase 2 结论

Ascend Phase 2 已围绕 910B 特性实现：

- slot batching；
- 16-slot continuous runner；
- prefix reuse；
- forced-token decode；
- slot/cache 状态管理；
- NPU eager decode。

结果：

串行 baseline：3466.2s 16-slot eager：152.1s 优化后的 16-slot：87.0s 127-task 最终结果：656.4s

这说明 Ascend batching 确实有效，但扩展到完整 127-task 后，收益明显低于 H100。

主要原因是：

- batch decode 仍包含较多 eager launch；
- prefill 没有完全转化为低成本的统一执行路径；
- GDN fallback 和 layout transform 仍然存在；
- 工具调用、slot admission、prefill 和 decode 之间仍有串行化；
- 没有经过完整端到端验证的低精度 GEMM 路径；
- workload 扩展后，长上下文和最长 agent chain 的尾部开销被放大。

Phase 2 结论：Ascend-specific batching 已获得明确收益，但目前尚未形成稳定高吞吐的系统级执行方案。

———

Phase 3 结论

Ascend Phase 3 已进行了：

- Ascend C/AddCustom 编译和运行验证；
- GDN/conv 局部替换；
- layer-0 correctness；
- NPU L2/L3 profile；
- layout 和权重路径实验。

但目前尚未形成完整 Ascend v3 engine。局部 Ascend C paired 16-task 结果为：

baseline：440.812s Ascend C 局部替换：462.559s 回归：约 4.9%

这说明局部 kernel 替换没有自动转化为端到端收益。主要原因是局部优化带来的收益被调用、状态搬运、layout transform 和其他路径开销抵消。

Phase 3 结论：当前完成的是局部算子和 runtime 可行性探索，还没有形成完整的 Ascend-native 自写引擎。

———

对 H100 的正确定位

H100 的结果说明：

- 减少 host launch 和同步有很高收益；
- batch graph、低精度 GEMM、GDN fast path 和融合执行可以显著降低单步成本；
- 自写引擎需要系统性地同时处理 cache、调度、prefill、decode 和 kernel。

但 H100 的具体 CUDA 实现不能直接作为 Ascend 的实现方案。Ascend 需要根据 CANN、NPU layout、Cube/GEMM、graph API 和实际 profile 选择自己的路径。

———

下一步计划

下一阶段不再以复刻 H100 的具体实现为目标，而是继续寻找适合 910B 的系统级方案：

1. 建立稳定的 Ascend-native baseline； 2. 评估 MindIE、ATB、vLLM-Ascend、ACLNN 等现成推理能力； 3. 优先解决 NPU graph、固定 shape/bucket 和低开销 batch decode； 4. 继续分析 GDN、layout transform、prefill 和 GEMM 的真实占比； 5. 只对 profile 证明的热点开发 Ascend C kernel； 6. 逐步构建适合 910B 的 slot、cache、prefill/decode 和工具调度模型； 7. 所有候选继续使用 single、16-task、127-task 三级验证，并检查 accuracy、token trajectory、turns、tool calls 和 tokens。